

Язык программирования j

RUSTовщики

ITMO

29 мая 2026 г.

- Исходный язык - Язык программирования `j` - императивный язык программирования со статической типизацией
- Целевой язык - Байт-код собственной стековой виртуальной машины
- Инструментальный язык - Rust
- Ожидаемый результат работы - работающий end-to-end компилятор и виртуальная машина

- Оператор `new`/литерал `null` - инициализация ссылочных типов
- `type coercion` - Оператор `::`
- Паники и `assert` - (Условное) аварийное завершение программы
- Массивы неизвестной длины - можно использовать не только в аргументах
- `Compile-time` константы - используются в статическом контексте (например, в типах)
- Фиксированное расширение файлов - e.g. `!my-program!`

Файл: ;type_aliases!

Листинг 1: Исходный код программы с псевдонимами типов

```
type kilometers is real;
type miles is real;

routine miles_to_kiometers(dist : miles)
  => ((dist :: real) * 1.60934) :: kilometers;

routine main() is
  var dist is 10.0 :: miles;
  var result : kilometers is miles_to_kiometers(dist);
  print result;
end
```

Файл: ;print!

Листинг 2: Исходный код программы с массивами

```
type recursive is record
    var id is integer;
    var link is recursive;
    var links is array [] recursive;
end;

routine main() is

    var root is new recursive where
        id is 0;
        links is new[3] array [] recursive;
    end;
    print root;

end;
```

Файл: `i.constant!`

Листинг 3: Исходный код программы с константами

```
constant LENGTH is 100;
constant DOUBLE_LENGTH : integer is LENGTH * 2;

type table is array[LENGTH * DOUBLE_LENGTH] integer;

routine meow(arr : array [LENGTH] real): real is
  assert arr.length = LENGTH;
  return arr[1];
end
```

Компоненты компилятора

- **Лексер** – разбивает исходный текст на лексемы и формирует токены.
- **Парсер** – по потоку токенов строит дерево разбора (parse tree).
- **Семантический анализатор** – преобразует дерево разбора в AST, выполняет проверки (типы, области видимости и др.), обогащает узлы.
- **Генератор байткода** – обходит аннотированное AST и порождает последовательность инструкций.

Промежуточные и выходные форматы данных

- **Единица трансляции** - один файл с исходным кодом
- **Поток токенов** – ленивая последовательность токенов от лексера.
- **Дерево разбора** – синтаксическая структура по грамматике языка.
- **AST** – Типизированное представление с разрешёнными именами.
- **Байткод** – последовательность инструкций целевой VM.
- **Бинарный файл** – набор байткодов функций и RTTI.



Лексический разбор

- **Входной формат** - Исходный текст программы
- **Выходной формат** – Ленивый поток токенов
- **Алгоритм** - Самописный разбор лексем

Лексический анализатор разбивает текст на лексемы, парсит целочисленные и вещественные литералы. Лексический анализатор всегда завершается успешно, нарушения лексической структуры возвращаются в виде `invalid` - токенов с информацией об ошибке. Поддерживает Unicode **синтаксический разбор**

- **Входной формат** - Поток токенов
- **Выходной формат** – Дерево разбора
- **Алгоритм** - Самописный рекурсивный спуск

Синтаксический анализатор производит дерево разбора из потока, отсекая синтаксически некорректные программы

Токены языка

Токен	Параметры	Описание
Identifier	имя: Identifier<'a>	Идентификатор
Keyword	ключевое слово: Keyword	Ключевое слово
Literal	значение: Literal	Литерал (число, true, false, null).
BuiltinType	имя: BuiltinType	Имя встроенного типа
Operator	оператор: Operator	Оператор
Comment	текст: Comment<'a>	Комментарий
Unexpected	символ: char	Ошибка лексера
Скобки		
Left/Right, Brace/Parenthesis	—	Парные токены скобок
RightArrow	—	Стрелка =>.
Cast	—	Приведение типа ::
Assignment	—	Присваивание :=.
RangeSymbol	—	Диапазон ...
Разделители		
Semicolon, Colon, Comma, Dot	—	Точка с запятой ;, двоеточие :, запятая ,, точка .

Семантический разбор

- **Входной формат** - Дерево разбора
- **Выходной формат** - AST
- **Алгоритм** - Однопроходный анализ

семантический анализатор:

- Разрешает имена
- Строит таблицы идентификаторов
- Производит анализ типов
- Производит вывод типов
- Разрешает ad hoc полиморфизм для операторов
- Вычисляет compile-time константы

Пример AST: f(new [3 + 3.0] array [] int, null)

```
Call
  f (3)
    NewArray
      integer
      RealToInt
        BinOp +
          IntToReal
            Literal 3 (Int)
            Literal 3.0 (Real)
    Null
```

Все идентификаторы (например, f) получили уникальный номер, добавлены приведения типа, проведена проверка типов

Интерпретатор

- **Входной формат** - AST, аргументы main
- **Выходной формат** - Выход программы

Рекурсивный интерпретатор, исполняющий AST, может использоваться вместо виртуальной машины и как эталон при её тестировании

Кодогенерация

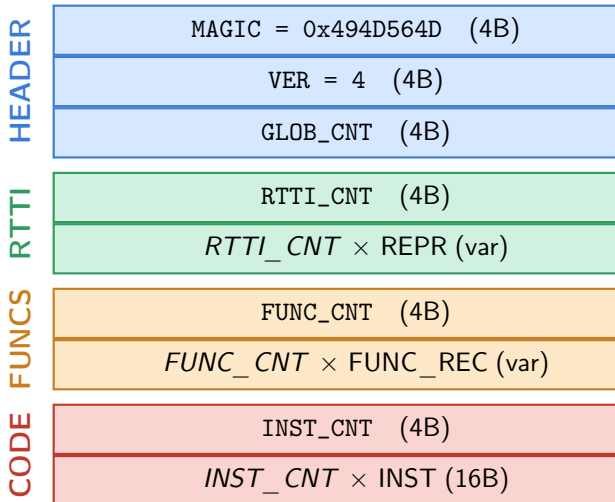
- **Входной формат** - AST
- **Выходной формат** - Бинарный формат

На этапе кодогенерации:

- Тела функций компилируются в байткод
- Собирается RTTI
- Собирается таблица функций
- Производится результирующий бинарный файл

Байткод собственной виртуальной машины:

- Стековая архитектура
- Три встроенных типа данных - целое, вещественное и указатель
- Автоматическое управление памятью
- Минифицированная информация о типах (форме данных) записывается в бинарник и используется в рантайме



Инструкции байткода (1/3)

Инструкция	Параметры	Описание
IntConst	value: Integer	Поместить целое значение на стек.
RealConst	value: Real	Поместить вещественное значение на стек.
NullConst	—	Поместить null на стек.
Load	loc: Location	Поместить на стек значение переменной.
Store	loc: Location	Снять значение со стека и записать в переменную.
AddressOf	loc: Location	Поместить на стек адрес переменной.
Dup	—	Дублировать вершину стека.
Drop	—	Удалить вершину стека.
DropMany	count: VarLoc	Удалить count элементов со стека.
Swap	—	Обменять местами два верхних элемента стека.

Инструкции байткода (2/3)

Инструкция	Параметры	Описание
BinOp	op: BinaryOperator	Применить операцию к значениям на вершине стека
UnOp	op: UnaryOperator	Применить операцию к значению на вершине стека
StoreAddress	—	Запись по косвенному адресу.
LoadAddress	—	Чтение по косвенному адресу
AllocRecord	type_id: Typeld, size: u64	Аллокация записи
AllocArray	type_id: Typeld, size: u64	Аллокация массива известного размера
AllocArrayDynamic	type_id: Typeld	Аллокация массива вычисленного размера
ArraySize	—	Вычисление длины массива
ElementAddress	—	Вычисление адреса элемента массива
FieldAddress	field_offset: u64	Вычисление адреса поля записи

Инструкции байткода (3/3)

Инструкция	Параметры	Описание
Label	id: u64	Метка
Jump	label: u64	Безусловный переход на метку.
JumpZero	label: u64	Условный переход на метку.
JumpNotZero	label: u64	Условный переход на метку.
Ret	—	Возврат из функции
Call	function_label: u64	Вызов функции по метке начала
Print	type_id: Typeld	Вызов built-in print
Panic	code: u64, line: u32, column: u16	Аварийное завершение с кодом ошибки.
IntToBool	—	Примитивное преобразование
RealToInt	—	Примитивное преобразование
IntToReal	—	Примитивное преобразование

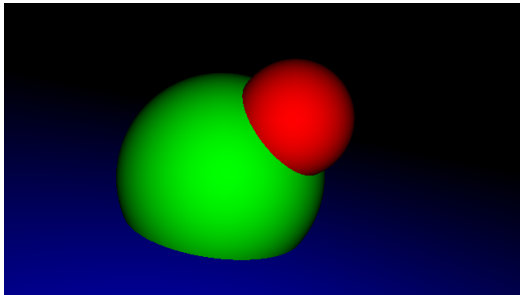
Виртуальная машина

- **Входной формат** - Бинарный файл с байткодом, имя функции, аргументы функции
- **Выходной формат** - Выход программы
- Инструментальный язык - C++
- Итеративный интерпретатор байткода
- Stop the world, Mark&Sweep сборщик мусора

Результаты

Компилятор и интерпретатор корректно работают на 100% тестов, представленных в репозитории, включая прикладные программы.

Виртуальная машина работает на 100% тестов, представленных в репозитории.



Изображение, полученное путём исполнения трассировщика лучей на языке j

Григоренко Павел

- Определение используемых структур
- CI
- Лексический анализатор
- Синтаксический анализатор
- Интерпретатор
- Кодогенерация - RTTI, бинарный формат

Попов Егор

- Определение используемых структур
- Лексический анализатор
- Синтаксический анализатор
- Семантический анализатор
- Кодогенерация - компиляция кода функций

Власенков Андрей

- Дизайн байткода
- Виртуальная машина - интерпретатор
- Виртуальная машина - сборщик мусора
- Тесты
- Кодогенерация - бинарный формат

Спасибо за внимание!

Репозиторий с исходным кодом